

LIBSPACE

Project Title : Libspace API - A Robust Backend for Library Operations.

LibSpace API is a backend library management system developed using Node.js, Express.js, and MongoDB. The project demonstrates secure user authentication and book management through RESTful APIs. It allows users to register, log in, and perform CRUD operations on books while ensuring data security using JWT authentication.

TEAM ID : SWTID-2026-8699

BY

TEAM LEADER : JAYDEN MACARIUS AMBROSE

TEAM MEMBER : LEREINA K A

TEAM MEMBER : SANTHOSH S

TEAM MEMBER : NARESH KUMAR P

TEAM MEMBER : VISHWA R

1. Project Overview

LibSpace is a backend library management system built using Node.js, Express.js and MongoDB.

It demonstrates authentication, REST API development and CRUD operations for managing books.

2. Project Instructions – LibSpace

1. Register a new user account using the registration API.
2. Log in using the registered credentials to receive an authentication token.
3. Use the token to access protected APIs.
4. Perform CRUD operations on books.
5. Logout securely when finished.
6. Users may also delete their account permanently.

Pre-requisites

Knowledge Requirements:

- JavaScript
- Node.js and Express
- REST APIs
- MongoDB and CRUD operations

Software Requirements:

- Node.js and npm
- MongoDB
- VS Code
- Thunder Client or Postman

3. Project Architecture

LibSpace follows a layered backend architecture consisting of:

1. API Layer (Routes)
2. Business Logic Layer (Controllers)
3. Data Access Layer (Models / Database)

This architecture ensures modular design, scalability, and maintainability.

Client → Routes → Controllers → Models → MongoDB

5. API Layer (Routes)

Routes handle incoming HTTP requests and map them to controllers.

User Routes:

POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
DELETE /api/auth/delete

Book Routes:

POST /api/book/add
GET /api/book/getBooks
GET /api/book/:id
PUT /api/book/:id
DELETE /api/book/deleteBook/:id

6. BusinessLogicLayer(Controllers)

Controllers contain the main application logic.

Responsibilities:

- User registration and login handling
- Password hashing using bcrypt
- JWT token generation
- CRUD operations for books
- Error handling and response formatting

7. DataAccessLayer(Models)

Models define MongoDB schemas.

User Schema Fields:

- name
- email
- password

Book Schema Fields: • url

- title
- author
- user (ObjectId reference)

8. DatabaseDesign

Database used: MongoDB

Collections:

1. Users
2. Books

Relationship:

One User → Multiple Books

Each book document stores a reference to the user who created it.

9. Security Features

- Password hashing using bcrypt
- JWT authentication
- Token verification middleware
- Token blacklist during logout
- Protected routes requiring authentication

10. Project Setup

1. Install dependencies

```
npm install
```

2. Create .env file

```
PORT=5050
```

```
MONGO_URI=mongodb://127.0.0.1:27017/libspace
```

```
JWT_SECRET=your_secret_key
```

3. Run the server

```
npm start
```

4. Access API

```
http://localhost:5050
```

11. API Testing

All APIs were tested using Thunder Client in Visual Studio Code.

User APIs:

```
POST /api/auth/register
```

```
POST /api/auth/login
```

```
POST /api/auth/logout
```

Book APIs:

```
POST /api/book/add
```

```
GET /api/book/getBooks
```

```
PUT /api/book/:id
```

```
DELETE /api/book/deleteBook/:id
```

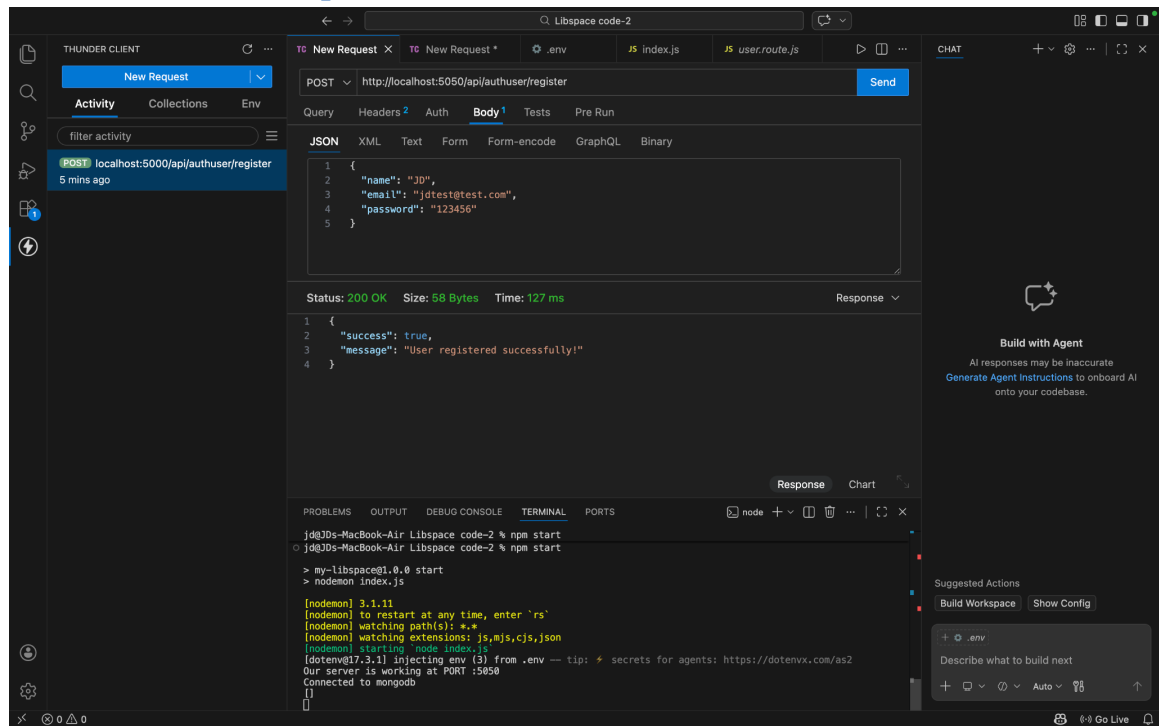
Features

- User Registration
- User Login
- Add Book
- Get Books
- Update Book
- Delete Book

Output Screenshots :

The following screenshots show successful API testing using Thunder Client.

Server Terminal Output



[illegible]

The screenshot displays the Thunder Client application interface. On the left, a sidebar shows a list of requests, including a POST request to 'localhost:5050/api/book/addbook' which was executed 'just now'. The main workspace is divided into two panes. The top pane shows the details of the selected request: a POST method to 'http://localhost:5050/api/book/addbook'. It includes tabs for Query, Headers, Auth, Body, Tests, and Pre Run. The 'Body' tab is active, showing a JSON payload: { "url": "www.libspacebook.com", "title": "Atomic Habits", "author": "James Clear" }. The bottom pane shows the response details: Status 200 OK, Size 33 Bytes, Time 51 ms. The 'Response' tab is active, displaying the JSON response: { "msg": "Book added successfully" }. On the right side of the application, there is a 'CHAT' panel with a 'Build with Agent' section, which includes a note about AI response accuracy and a 'Generate Agent Instructions' button. Below this, there are 'Suggested Actions' like 'Build Workspace' and 'Show Config', and a 'Describe what to build next' input field. The bottom status bar shows 'Response' and 'Chart' tabs, and the system tray at the very bottom includes icons for network, volume, and the 'Go Live' button.

THUNDER CLIENT

New Request

ActivityCollectionsEnv

filter activity

GETlocalhost:5050/api/book/getBooksjust now

POSTlocalhost:5050/api/book/addbookjust now

POSTlocalhost:5050/api/authuser/loginjust now

POSTlocalhost:5000/ap/authuser/register5 mins ago

Libspace code-2

New RequestNew RequestNew RequestNew Request X.env

GEThttp://localhost:5050/api/book/getBooksSend

QueryHeaders3AuthBodyTestsPre Run

HTTP HeadersRaw

Accept*/

User-AgentThunder Client (https://www.thunderclient.com)

AuthorizationBearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpZVCJ9.eyJpZCI6I

headervalue

Status: 200 OKSize: 154 BytesTime: 14 ms

ResponseHeaders6CookiesResultsDocs

```
1 {
2   {
3     "id": "69a9a1eda56393e81ce0f513",
4     "url": "www.libspacebook.com",
5     "user": "69a99eb8a56393ed1ce0f50d",
6     "title": "Atomic Habits",
7     "author": "James Clear",
8     "_v": 0
9   }
10 }
```

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

Suggested Actions

Build WorkspaceShow Config

+ .env

Describe what to build next

+⌵⌴⌵Auto⌵

ResponseChart

Get Book by ID

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

GET localhost:5000/api/book/69a9a1eda5... just now

GET localhost:5000/api/book/getBooks just now

POST localhost:5000/api/book/addbook just now

POST localhost:5000/api/authuser/login just now

POST localhost:5000/api/authuser/register 5 mins ago

GET http://localhost:5000/api/book/69a9a1eda56393e81ce0f513

Query Headers Auth Body Tests Pre Run

HTTP Headers

Accept */*

User-Agent Thunder Client (https://www.thunderclient.com)

Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImFkbG93IiwiaWF0IjoxNjU0MjU0MDAwfQ.eyJpZCI6ImFkbG93IiwiaWF0IjoxNjU0MjU0MDAwfQ

header value

Status: 200 OK Size: 172 Bytes Time: 16 ms

Response Headers Cookies Results Docs

```
1 {
2   {
3     "id": "69a9a1eda56393e81ce0f513",
4     "url": "www.libspace-book.com",
5     "user": "69a99eb8a56393e81ce0f50d",
6     "title": "Atomic Habits - Updated Edition",
7     "author": "James Clear",
8     "_v": 0
9   }
10 }
```

Copy

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

Suggested Actions

Build Workspace Show Config

+ .env

Describe what to build next

+ Auto

Update Book

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

PUT localhost:5000/api/book/69a9a1eda5... just now

GET localhost:5000/api/book/69a9a1eda5... just now

GET localhost:5000/api/book/getBooks just now

POST localhost:5000/api/book/addbook just now

POST localhost:5000/api/authuser/login just now

POST localhost:5000/api/authuser/register 5 mins ago

PUT http://localhost:5000/api/book/69a9a1eda56393e81ce0f513

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "title": "Atomic Habits - Updated Edition"
3 }
```

Format

Status: 200 OK Size: 25 Bytes Time: 39 ms

Response Headers Cookies Results Docs

```
1 Data updated successfully
```

Copy

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

Suggested Actions

Build Workspace Show Config

+ .env

Describe what to build next

+ Auto

Delete Book

The screenshot shows the Thunder Client interface with a DELETE request configured. The URL is `http://localhost:5050/api/book/deleteBook/69a9a7cba56393e81ce0f52c`. The request is sent from the Thunder Client, and the response is a 200 OK status with a JSON body: `{ "msg": "Book deleted successfully" }`. The interface includes a sidebar with a list of requests, a main panel for request details, and a right sidebar with a chat area and suggested actions.

Request Details:

- Method: DELETE
- URL: `http://localhost:5050/api/book/deleteBook/69a9a7cba56393e81ce0f52c`
- Status: 200 OK
- Size: 35 Bytes
- Time: 8 ms

Response:

```
{
  "msg": "Book deleted successfully"
}
```

User Logout

The screenshot shows the Thunder Client interface with a POST request configured. The URL is `http://localhost:5050/api/authuser/logout`. The request is sent from the Thunder Client, and the response is a 200 OK status with a JSON body: `{ "msg": "User has logged out successfully" }`. The interface includes a sidebar with a list of requests, a main panel for request details, and a right sidebar with a chat area and suggested actions.

Request Details:

- Method: POST
- URL: `http://localhost:5050/api/authuser/logout`
- Status: 200 OK
- Size: 42 Bytes
- Time: 7 ms

Response:

```
{
  "msg": "User has logged out successfully"
}
```

Conclusion

LibSpace demonstrates secure backend development using Node.js, Express.js and MongoDB.

